

KineticCare

Database Schema & Architecture Technical Specification

Project: Healthcare Appointment Platform

Developer: Mihir Parida

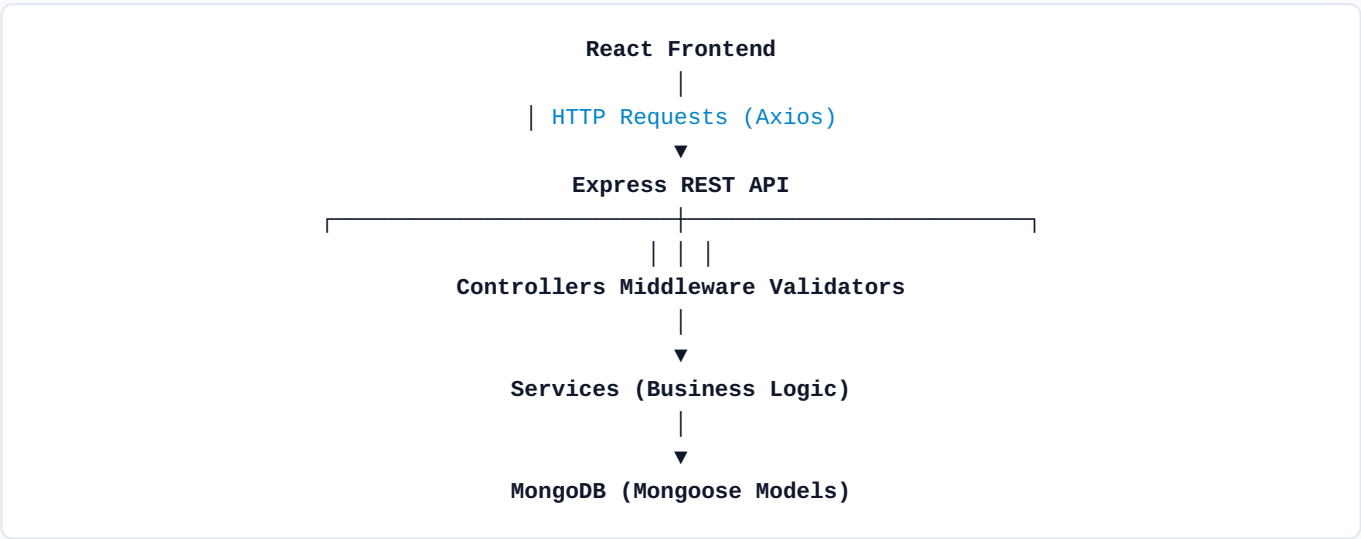
Stack: MERN (MongoDB, Express, React, Node)

1. Project Overview

KineticCare is a full-stack MERN healthcare appointment booking application designed to streamline patient-provider interactions. The platform enables users to browse specialized healthcare services, reserve appointment slots, manage bookings, and authenticate seamlessly using standard Email/Password credentials or Google OAuth. Additionally, an administrative dashboard provides controls for managing services, schedule slots, and aggregate user bookings.

2. System Architecture

The application adopts a clean, 3-layer architecture pattern to maintain strong separation of concerns, scalability, and ease of testing.



3. Application Structure

Frontend Architecture

Frontend

- └─ Components # Reusable UI elements
- └─ Pages # Full application screens
- └─ Layouts # Public & Admin wrappers
- └─ Routes # Protected/Public routing
- └─ Context # Auth state management
- └─ Hooks # Custom React hooks
- └─ Services # Axios API calls
- └─ Utils # Helper functions
- └─ Assets # Static files & styles

Backend Architecture

Backend/src

- └─ controllers # HTTP Handlers
- └─ services # Business logic
- └─ models # Mongoose schemas
- └─ routes # Express endpoints
- └─ middleware # Auth & Errors
- └─ validators # Express Validator
- └─ utils # Helpers / Helpers
- └─ config # DB & OAuth config
- └─ server.js # Entry point

4. Database Design & Schemas

MongoDB serves as the primary data store, using four core collections to encapsulate platform operations.

User Collection

_id	ObjectId (PK)
name	String
email	String (Unique)
password	String (Hashed)
avatar	String
role	String (User/Admin)
googleId	String
refreshToken	String
createdAt	Date
updatedAt	Date

Purpose: Stores authentication, authorization, and profile data for end users and administrators.

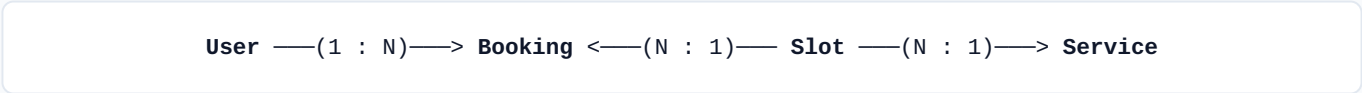
Service Collection

_id	ObjectId (PK)
name	String
category	String
description	String
duration	Number (Mins)
price	Number
image	String (URL)
isActive	Boolean
createdAt	Date
updatedAt	Date

Examples: Physiotherapy, Orthopedic Consultation, General Checkup.

Slot Collection		Booking Collection	
<code>_id</code>	ObjectId (PK)	<code>_id</code>	ObjectId (PK)
<code>service</code>	ObjectId (Ref)	<code>user</code>	ObjectId (Ref)
<code>date</code>	Date	<code>service</code>	ObjectId (Ref)
<code>startTime</code>	String	<code>slot</code>	ObjectId (Ref)
<code>endTime</code>	String	<code>bookingStatus</code>	String
<code>maxBookings</code>	Number	<code>paymentStatus</code>	String
<code>availableBookings</code>	Number	<code>notes</code>	String
<code>createdAt</code>	Date	<code>createdAt</code>	Date
<code>updatedAt</code>	Date	<code>updatedAt</code>	Date
<i>Purpose:</i> Defines specific appointment times and tracks dynamic seat capacity.		<i>Purpose:</i> Stores appointment bookings made by users and links User, Service, and Slot.	

5. Entity Relationships



- **User to Booking:** One User can create multiple Bookings (1:N).
- **Service to Slot:** One Service can offer multiple time Slots (1:N).
- **Slot to Booking:** One Slot can host multiple Bookings up to its `maxBookings` capacity (1:N).

6. Core Application Workflows

Authentication Flow

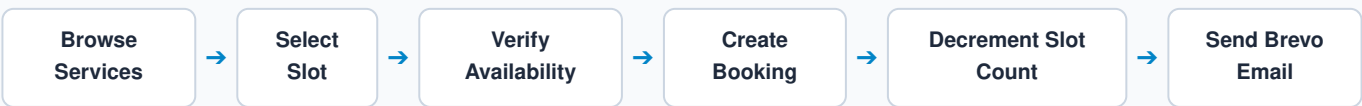
Email Login:



Google OAuth Login:



Booking Flow



7. Technical Stack & Security Implementation

Technology Stack

Frontend:

React Vite Tailwind CSS React Router

TanStack Query Axios React Hook Form

Backend & Database:

Node.js Express.js MongoDB Mongoose

JWT Google OAuth Brevo Email API

Security Measures

- **JWT Authentication:** Secure token generation and expiration handling.
- **HTTP-only Cookies:** Mitigates XSS attacks by keeping tokens out of JS access.
- **Password Hashing:** Strong `bcrypt` encryption before storing credentials.
- **Role-Based Access Control:** Strict separation of User and Admin permissions.
- **Helmet & Validator:** Hardens HTTP headers and sanitizes incoming body payloads.

8. Architectural Design Decisions

- **Layered Architecture:** Strict Routes → Controllers → Services → Models isolation enforces separation of business logic and request handling.
- **Document Storage:** MongoDB was selected because patient appointments, slot schedules, and profiles fit well into flexible, JSON-like document structures.
- **State Management:** TanStack (React) Query streamlines server-state management, eliminates redundant API calls, and automatically syncs data caches.
- **Hybrid Authentication:** Integrating Google OAuth along with local email/password provides zero-friction onboarding for end users.